Goosebot Battery Controller

Richard Carter

The software, design, and related materials shown here is available on Github at https://github.com/bax2004rj/bms

Abstract

Failures due to battery voltage loss on Goosebots have been a major issue, as these robots have no way to measure battery voltage while running. A device to monitor voltages and provide a communication vector and automatic shutoff device are discussed.

Introduction

On many battery-powered electronic devices, battery management systems are used to control the power delivered to the device [2]. These systems monitor battery voltage to display state of charge, and often have protection circuitry built in to protect the batteries or the device from being damaged. In the case of this situation, Goosebots presently have no such circuitry, and suffer from numerous issues caused by critically low battery voltage. In this report, a circuit is built and demonstrated which aims to resolve these issues.

Materials and Method

Materials

The materials used are as follows:

Item	Quantity
Arduino Nano	1
Buck Converter	1
MOSFET (IRF3205)	2
7 Segment Display (E30401)	1
Opamp (LM358)	1
Resistor (1Ω)	1
Resistors (220Ω)	3
Resistor (1KΩ)	1
Resistor (2KΩ	1

Table 1. Bill Of Materials

Design

The design was built to monitor the voltage using a voltage divider, monitor current by way of a current shunt (which was not fully realized in software), act as a step down for the on-board computer, communicate with the on-board computer, and feature automatic switching [1]. The design of the controller followed the following block diagram:

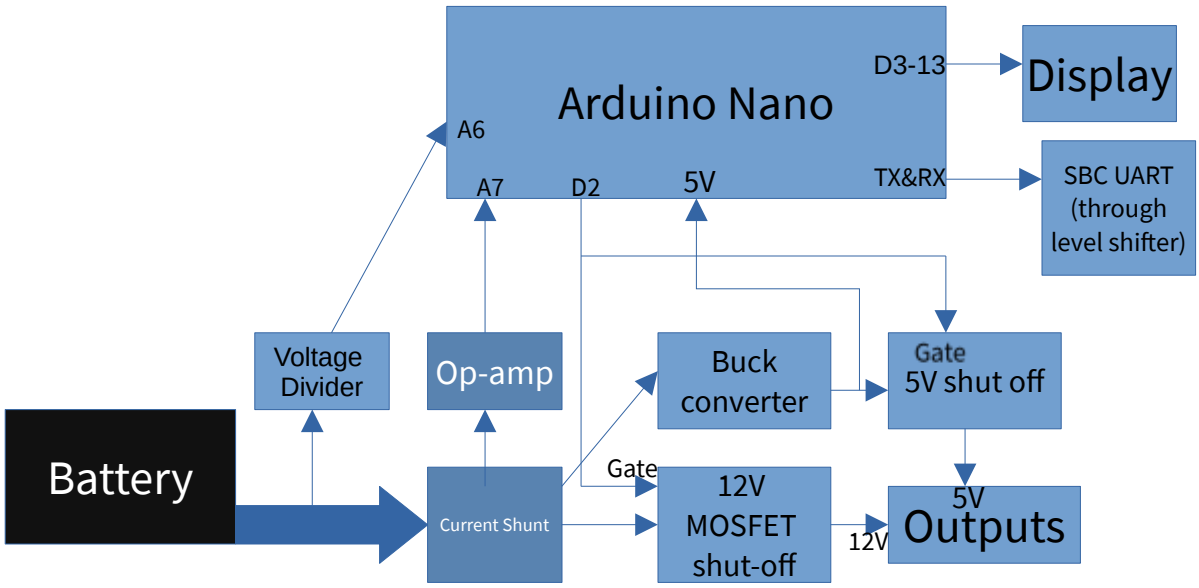

Figure 1. Circuit Block Diagram

Using KiCad, the design was made into the following schematic:

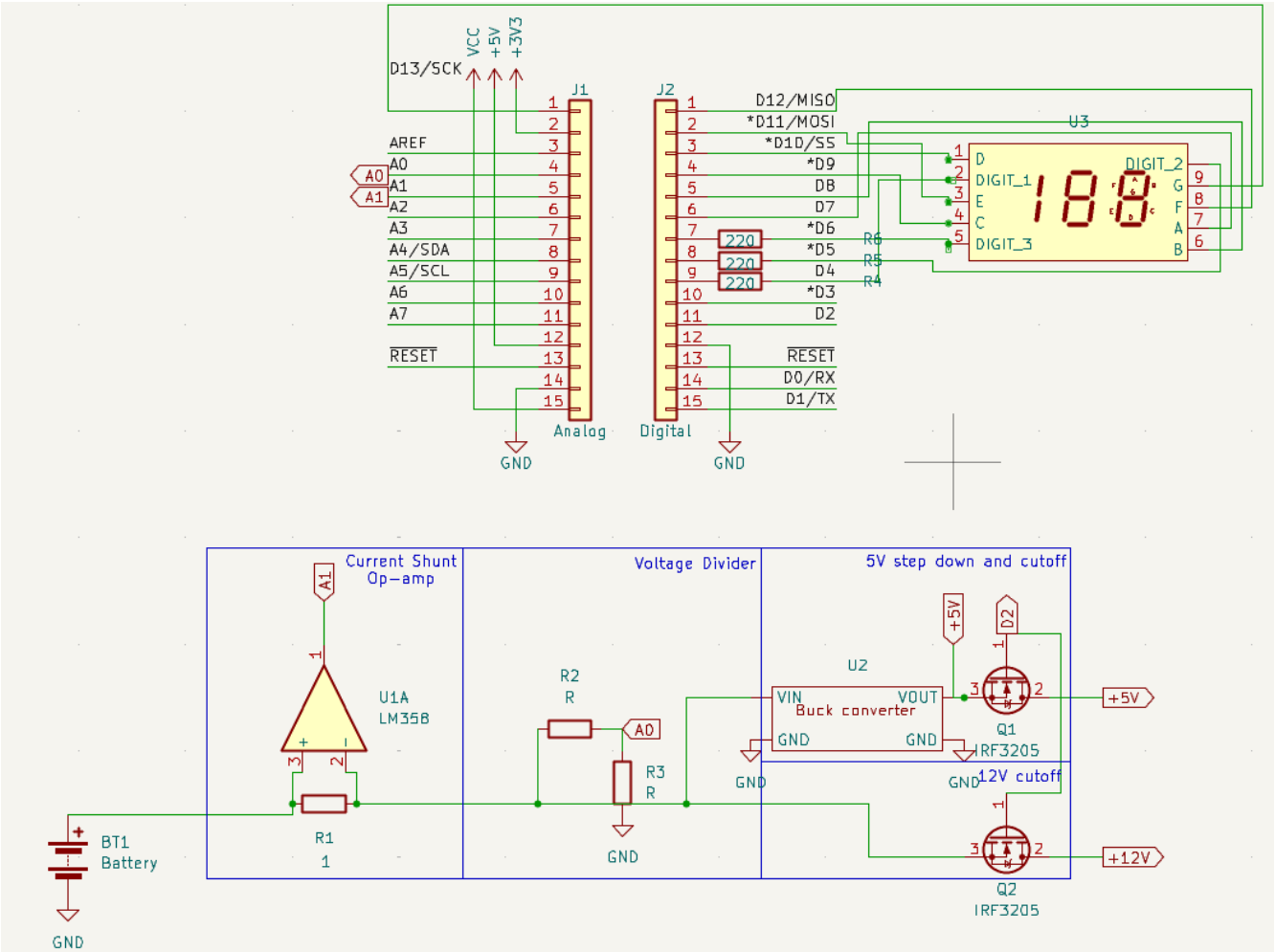

Figure 2. Circuit Schematic

This design was then constructed on a breadboard.

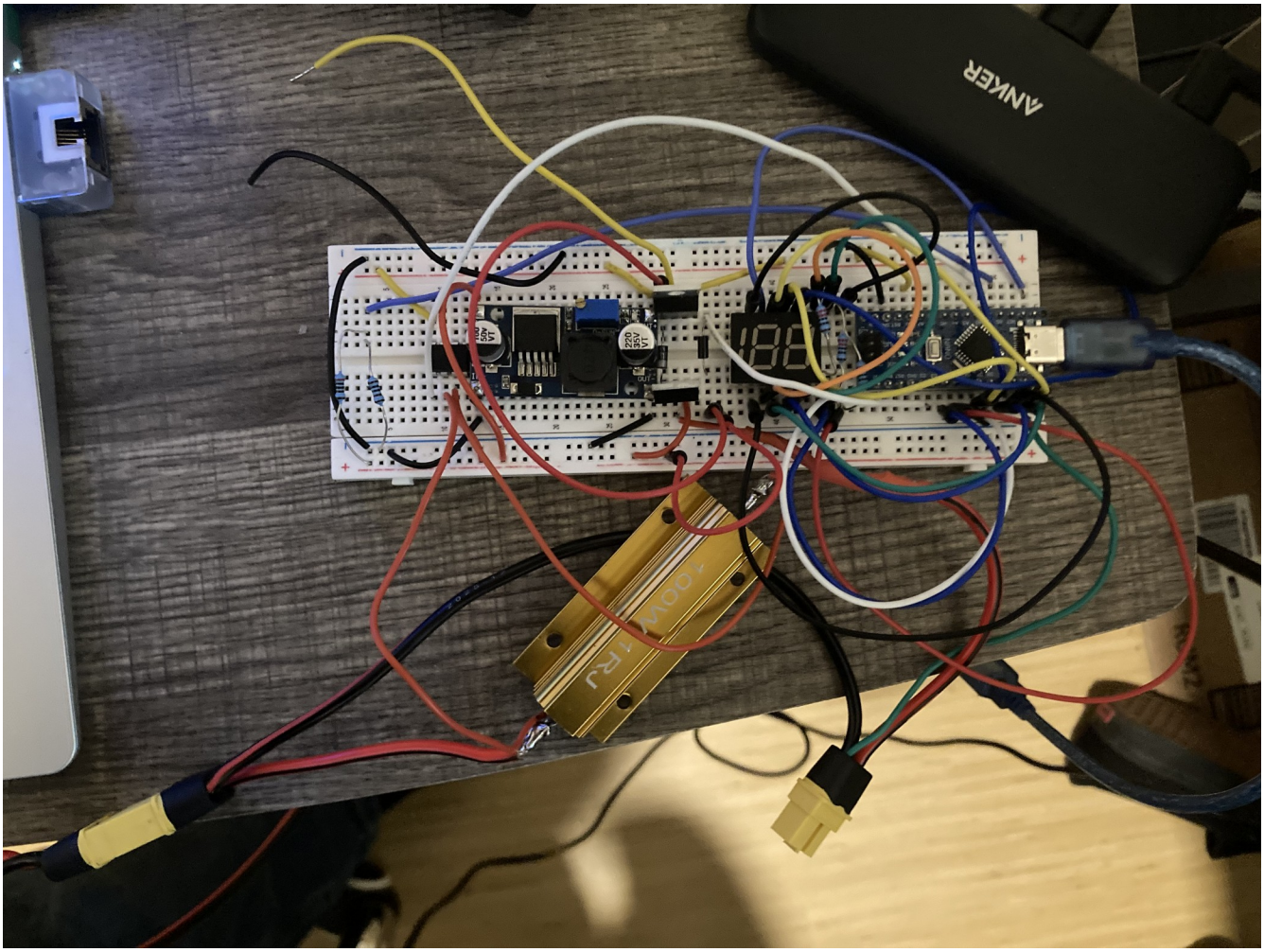

Figure 3. Breadboard Circuit

Testing

Since testing on an actual Goosebot could not be done in time, the behavior of a LiPo battery was simulated using a DC power supply. The voltage was varied from 10V to 13V and the measured voltage and state of charge were noted from the serial communication functionality. This data was obtained over serial via a computer running the Arduino IDE.

To test the low-battery switching functionality, the 12V output was probed with a multimeter and the voltage was varied between just below the 10V (0% state of charge [3]) threshold, and above the 10V threshold. This test was repeated probing the voltage at the output from the Arduino.

Other functionality, such as the current monitoring could not be tested, as the firmware did not have the capability to successfully measure current at the time of testing.

Results

Voltage and state of charge

The voltage results were recorded from the multimeter and Serial Monitor in the below table.

Input Voltage (V)	Arduino Reported Voltage (V)	Reported State of Charge (%)
9.95	9.60	0
11.6	11.2	49
13.0	13.13	100

Table 2. Battery Voltage vs. Reported Voltage vs. State of Charge

Switching Circuit

The automatic low battery switching was tested, and the voltage measurements were recorded into the following table.

Input Voltage (V)	Output Voltage (V)	Arduino pin voltage (V)
11.5	7.936	4.835
9.88	8.339	0.98

Table 3. Switching Voltage test results

Discussion

Based on the results of Table 2, the Arduino was able to correctly read out the state of charge of the battery. The voltage measured was approximately off by 0.3-0.5V in the tests shown here, which did affect the state of charge readings. This may affect the perceived battery life by as much as 1%, as the circuit may report a slightly lower battery voltage than usual. It was also noted on tests not recorded here that the difference between the Arduino voltage and the power supply voltage varied at times. This error can be corrected by averaging voltage measurements.

Based on Table 3, it can be concluded that the MOSFET switching function was not functional, holding the voltage high at all times. The Arduino was successfully switching on as needed, however the MOSFET did not get properly driven, leading to the fault. Future designs should be tested using a MOSFET gate driver, to properly switch the MOSFET.

Conclusions

It can be concluded that the demonstrated circuit was able to successfully measure the voltage and state of charge of a LiPo battery, and communicate state of charge with an on-board computer. The switching functionality was not functional, but future circuit implementations could make this system functional.

Overall, the voltage monitoring and computer communication systems were successfully implemented and achieved.

Acknowledgment

The author would like to thank Dr. Hoan Ngo for the opportunity to work on this project and offering insight on the design of Goosebots. Additionally, the author would also like to thank various friends and colleagues within Dr. Ngo's Autonomous Robotic Systems course, who offered insight about the design of Goosebots as well the robot's specifications.

References

[1] Carter, R. "Goosebot BMS Board Idea", April 2026.

[2] Husain, I. "Electric and Hybrid Vehicles Design Fundamentals", 3rd ed. Boca Raton, FL, USA: CRC Press, 2021.

[3] Beech, J. "The care and feeding of LiPo packs", Promodeler. https://www.promodeler.com/askJohn/How-To-Care-for-LiPo-Packs (accessed April 15, 2026)